

Architecting Distributed LLM Training on Consumer Hardware: A Guide to CPU-Only Nanochat Forks for LAN and WAN

Architectural Blueprint for a Minimalist yet Distributed System

The primary objective is to create a fork of `nanochat` that democratizes Large Language Model (LLM) training by enabling it to run on distributed, low-resource, CPU-only machines connected via Local Area Networks (LAN) and Wide Area Networks (WAN). The original `nanochat` repository is celebrated for its minimalist design, comprising approximately 8,000 lines of code that cover the entire LLM training pipeline—from tokenization to inference—in a single, dependency-light codebase [51](#) [69](#). Its architecture is fundamentally a Transformer model, conceptually similar but simpler than models like Llama [28](#) [41](#). The central challenge in achieving the user's goal is to augment this minimalist core with sophisticated distributed computing capabilities without compromising its clarity and accessibility. The proposed solution involves implementing a hybrid strategy of data parallelism and model sharding, which necessitates significant changes to the core training loop for coordination and communication [20](#) [72](#).

A critical design decision revolves around how to integrate these new features. Deeply embedding complex coordination logic directly into the existing `nanochat` training script would likely violate its core philosophy of simplicity and could make the codebase difficult to maintain and modify [51](#). A more robust and scalable approach is to adopt a modular architecture, separating the core training logic from the distributed orchestration layer. This separation of concerns allows the distributed components—such as the task scheduler, heartbeat mechanism, and gradient aggregation protocol—to be developed and maintained independently. For instance, one could envision a "manager" or "coordinator" process that oversees the distributed training job, while individual worker processes execute the actual training steps. This structure mirrors principles seen in larger distributed systems like Ray, where a central component manages a pool of workers [43](#). The coordinator would be responsible for tasks such as distributing data batches, aggregating results, and managing the state of the worker nodes. This architectural pattern is particularly well-suited for a project aiming to build upon a small,

focused codebase, as it allows for incremental development. One could start with a simple master-worker model and evolve it towards a more resilient peer-to-peer or leaderless architecture as the project matures [16](#) [18](#).

The choice of programming language and libraries will be pivotal. Since `nanochat` is written in Python, leveraging the extensive Python ecosystem for distributed computing is a natural path. Libraries like PyTorch provide foundational tools for multi-machine training through its Distributed RPC framework, which offers primitives for remote communication [88](#). However, PyTorch's default collective communication backends like NCCL are GPU-centric and unsuitable for this CPU-only requirement [80](#). Therefore, any implementation must either use PyTorch's CPU-based collective operations or develop a custom communication layer using standard Python libraries. For networking, a lightweight Remote Procedure Call (RPC) library could serve as the backbone for inter-node communication between the manager and workers. Python libraries such as `tinyrpc` offer a modular framework where transport (e.g., TCP sockets), protocol (e.g., JSON, MessagePack), and dispatching can be configured independently, providing the flexibility needed to build a bespoke, efficient communication layer [93](#). Similarly, defining service contracts and data structures using Protobuf with a gRPC implementation provides a strong foundation for building robust services [94](#). While gRPC introduces more overhead than raw sockets, it offers benefits like built-in serialization, interface definition, and client-side retry mechanisms, which can simplify the development of fault-tolerant applications [95](#).

Another key consideration is the integration of the requested HTML dashboard. This feature requires a web server component to be part of the system. To maintain the minimalist ethos, a micro-framework is essential to avoid the heavy overhead of full-stack solutions like Django [31](#). Flask is an excellent candidate due to its minimalist design and ease of integration [31](#) [32](#). FastAPI is another powerful option, offering high performance through asynchronous request handling and automatic API documentation generation, making it ideal for I/O-bound tasks like serving dashboard updates [78](#). Bottle is also a viable single-file micro-framework known for its extremely low memory footprint, making it suitable for resource-constrained environments [48](#). The chosen framework will host the static HTML, CSS, and JavaScript files for the dashboard UI and provide the backend endpoints for fetching training metrics and status information. This creates a clear separation between the computational workloads (training) and the monitoring workload (the dashboard server).

Finally, the overall system architecture must be designed for decentralization. Instead of relying on a single point of failure, such as a central master node, the system should

incorporate mechanisms for peer discovery and decentralized control. This is a common theme in research on decentralized federated learning and edge computing, where nodes operate in an environment with no guaranteed central authority [60](#) [62](#). For example, nodes could periodically broadcast their availability and status to a multicast address, allowing other nodes to discover and form a cluster dynamically. When a new node joins or an existing one leaves, the remaining nodes can detect this change and adjust their behavior accordingly. This peer-to-peer approach enhances resilience and scalability, which are paramount for a system intended to run on a collection of personal, potentially unreliable machines. The architecture must therefore include components for node discovery, state management, and dynamic reconfiguration of the training job in response to changes in the cluster topology. This design philosophy is crucial for ensuring the long-term viability and practicality of the forked project in the target deployment scenarios.

Communication Protocol Design for Unreliable Networks

Designing an effective communication protocol is arguably the most challenging aspect of creating a distributed LLM training system for LAN and WAN environments. Unlike a homogenous data center where high-speed, low-latency networks like RDMA are common, this project targets heterogeneous, consumer-grade hardware connected over potentially unstable and bandwidth-constrained networks [68](#) [79](#). Network latency, packet loss, and variable bandwidth are not edge cases but expected characteristics of the operational environment [19](#). Therefore, the communication layer must be resilient, efficient, and capable of gracefully degrading under adverse conditions. The design must differentiate between various types of communication flows within the system: real-time monitoring data from the training nodes to the dashboard, periodic health checks between nodes, and the bulk transfer of model gradients during synchronization.

For real-time monitoring, the choice of protocol is critical for both performance and user experience. While WebSockets offer a persistent, bidirectional communication channel, they may be overkill for a simple monitoring dashboard where data primarily flows from the server (the training node or manager) to the client (the browser) [6](#) [30](#). Server-Sent Events (SSE) are explicitly designed for this unidirectional stream of data and are generally more bandwidth-efficient because they leverage standard HTTP connections and only transmit data when events occur, leading to a smoother traffic profile [3](#) [6](#). Benchmarks comparing SSE and WebSockets often show them to be comparable in

performance if properly implemented, but SSE is conceptually simpler for server-to-client streaming tasks [91](#). For updating charts with loss values, throughput metrics, or node status, SSE provides a perfect balance of efficiency and functionality, avoiding the overhead of maintaining a full-duplex WebSocket connection unless interactive commands from the dashboard are required [77](#). Long-polling, an older technique used in Comet-style applications, is significantly less efficient as it involves repeated HTTP requests and does not provide a persistent stream, making it unsuitable for this use case [4](#) [47](#).

For internal coordination between nodes, a lightweight Remote Procedure Call (RPC) framework is the most appropriate solution. The requirements here are reliability, low overhead, and structured communication. While complex message queues like Apache Kafka offer powerful fault-tolerant message storage, their setup and maintenance might be excessive for this project's scope [89](#). A simpler, custom-built RPC layer over standard TCP sockets provides greater control and efficiency. The Python library `tinypc` is an excellent choice as it decouples the transport mechanism from the application logic, allowing developers to experiment with different serialization formats (like MessagePack for efficiency) and transport layers [93](#). This modularity is key to adapting the communication layer to different network conditions. The RPC calls would handle critical functions such as:

- **Heartbeats:** Regular signals sent between nodes to detect failure or churn. A failure to receive a heartbeat indicates a node may have gone offline.
- **Task Assignment:** The manager assigning a batch of data to a free worker for training.
- **Result Reporting:** Workers reporting their computed gradients or loss values back to the manager for aggregation.
- **Node Registration:** New nodes announcing their presence and available resources to join the cluster.

The most communication-intensive operation in a distributed training setup is the synchronization of gradients in a data parallelism scheme. In a synchronous approach, all workers must wait for each other at a barrier before proceeding, which can be severely bottlenecked by the slowest node or the highest network latency [26](#). To mitigate this, several advanced techniques can be employed. One promising method is infrequent synchronization, where workers train for multiple local steps before communicating and averaging their gradients [27](#). This reduces the frequency of expensive network operations and helps mask network latency. Another highly relevant technique, inspired by the SparseLoCo method, is to exchange sparse pseudo-gradients instead of dense ones [27](#).

This drastically reduces the volume of data transferred over the network, which is particularly beneficial for WAN training where bandwidth is limited [58](#). Furthermore, subspace pipeline compression, which involves compressing activations and gradients between stages of a pipelined model, has been shown to be effective in reducing bandwidth usage with only a modest impact on model loss [27](#). Implementing such communication-saving strategies is not just an optimization but a necessity for making distributed training feasible on the target low-bandwidth networks.

Implementing Hybrid Parallelism: Data Sharding and Model Partitioning

To scale LLM training beyond the memory and processing limits of a single machine, the fork must implement a hybrid parallelism strategy combining data parallelism and model sharding (also known as tensor or model parallelism). These two techniques address different scaling bottlenecks. Data parallelism is effective when the entire model can fit within the RAM of a single worker node, but the dataset is too large for one machine to process efficiently. Model sharding becomes necessary when the model itself is so large that it cannot fit on a single node's memory, even if there is sufficient disk space [20](#). The user's request for both strategies implies a need for a flexible framework that can adapt to different problem sizes and hardware configurations. The implementation must carefully manage the distinct communication patterns and complexities associated with each method.

Data parallelism is the more straightforward of the two strategies to implement. In this paradigm, an identical copy of the entire model is loaded into the memory of each participating worker node. The training dataset is partitioned, and each worker trains on its assigned subset of the data. After each forward and backward pass, the workers must synchronize their locally computed gradients. This is typically done by aggregating (e.g., averaging) the gradients from all workers to compute a single update step for the global model [72](#). The primary challenge in data parallelism is the All-Reduce operation, which incurs significant communication overhead, especially as the number of workers grows or network conditions deteriorate. As previously discussed, mitigating this overhead is critical. Techniques like infrequent synchronization, where workers perform several local updates before synchronizing, can dramatically reduce the frequency of these costly communication rounds [27](#). Another approach is to use compressed or sparse gradients, further minimizing the data that needs to be communicated across the network [27](#). For a

CPU-only environment, this strategy is highly viable for moderately sized models that can fit in the combined RAM of the available machines.

Model sharding, on the other hand, is far more complex. It involves partitioning the model's parameters and computation graph across multiple nodes. For a Transformer-based model like nanochat's, this could mean placing different layers on different machines or splitting the weight matrices of linear layers across several workers [20](#). During a forward pass, activations computed on one node must be sent to another node for the next stage of computation. Similarly, gradients from the backward pass must be propagated back through the chain of nodes. This creates a tightly coupled dependency between the workers, and any delay or failure in one node can halt progress for the entire model. The communication overhead is substantially higher than in data parallelism because data (activations and gradients) must flow through the model's execution graph, not just to a central aggregator.

To manage this complexity, the implementation should abstract away the distributed nature of the computation. A potential approach is to create a custom PyTorch module, perhaps called `ShardedTransformer`, which inherits from `nn.Module`. This module would contain the logic for partitioning the model and orchestrating the inter-node communication. When a forward pass is initiated, the module would determine which node holds the current layer's weights. If the data is not on the correct node, it would be serialized and sent over the network. After the computation, the result (activation) would be sent to the node hosting the next layer. This process would continue until the final output is produced. Backpropagation would follow the reverse path. This level of abstraction shields the main training loop from the intricacies of distributed execution, similar to how advanced frameworks like DeepSpeed or Alpa manage complex parallelism strategies transparently [43](#). Research into learning-based sharding strategies, where a reinforcement learning agent jointly optimizes parallelism degrees and sharding dimensions, points toward future avenues for optimization, though a simpler, statically defined partitioning scheme would be a practical starting point [21](#).

The following table compares the two parallelism strategies in the context of the proposed project:

Feature	Data Parallelism	Model Sharding
Memory Requirement per Node	Each node must hold a complete copy of the model 72 .	Each node holds only a portion of the model's parameters 20 .
Primary Bottleneck	Communication cost of synchronizing gradients (All-Reduce) 27 .	Latency of passing activations and gradients between nodes 20 .
Communication Pattern	Synchronous, all-to-all communication at each synchronization step.	Asynchronous, pipeline-like flow along the model's computation graph.
Resilience to Node Failure	High; failure of one worker only loses a fraction of the batch.	Low; failure of one worker halts the entire forward/backward pass.
Implementation Complexity	Moderate; requires a collective communication primitive.	High; requires careful orchestration of data movement between nodes.
Suitability for Target HW	Good for moderately sized models fitting in RAM of multiple slow machines.	Essential for very large models that exceed the capacity of any single machine.

Given these trade-offs, a phased implementation is recommended. First, focus on a stable, efficient implementation of data parallelism. Once this is working reliably, extend the framework to support model sharding. This incremental approach allows for the gradual addition of complexity while ensuring that each component is thoroughly tested. The initial version could even allow the user to specify a "parallelism mode" (data, model, or hybrid) at runtime, providing flexibility for different use cases and hardware setups.

Ensuring Fault Tolerance and Resilience in Decentralized Environments

Training LLMs is a resource-intensive process that can span days or weeks, making it highly susceptible to interruptions from hardware failures, software bugs, or network issues. In a centralized, controlled data center environment, fault tolerance is often managed with dedicated infrastructure like high-availability clusters and shared, reliable storage for checkpoints [63](#). However, the target deployment scenario—a decentralized network of low-resource, consumer-grade machines on a LAN or WAN—lacks this luxury. Nodes can go offline unexpectedly ("churn"), network connectivity can be intermittent, and there is no guarantee of persistent, shared storage [65](#). Therefore, designing a robust fault tolerance mechanism is not an optional feature but a fundamental requirement for the success of this project. The system must be able to detect failures, recover from them gracefully, and continue training with minimal disruption.

A naive fault tolerance strategy, such as frequent, synchronous checkpointing of the entire model state, is ill-suited for this environment. Global checkpointing requires all workers to pause their computation and write their state to a shared location simultaneously, which can introduce significant I/O bottlenecks and is difficult to implement in a truly decentralized manner [81](#). Moreover, in a WAN setting, writing to a central storage location can be prohibitively slow and unreliable [65](#). An alternative approach is to rely on the underlying cloud infrastructure's fault tolerance mechanisms, such as those offered by spot instances, but this again assumes a cloud environment and is not applicable to a self-managed home lab [64](#).

A more appropriate strategy for this project is to adopt an asynchronous, lightweight, and decentralized approach to fault tolerance. Instead of global checkpoints, each worker node should be responsible for saving its own local state periodically. This lazy, asynchronous checkpointing reduces I/O contention and allows nodes to recover independently [81](#). If a node fails, it can be restarted, and it can resume training from its last locally saved checkpoint. The challenge then shifts to resynchronizing the failed node with the rest of the cluster. This can be achieved by having the surviving nodes continue training and periodically broadcasting their latest model state. When the failed node rejoins the cluster, it can fetch the latest global model and resume its work from there. This model of recovery without relying on a single, globally-consistent checkpoint is a key principle in decentralized systems [64](#).

The system must also incorporate mechanisms for failure detection. A simple and effective method is a heartbeat protocol. Each active node sends a regular "heartbeat" signal (e.g., every few seconds) to a designated manager or to all other peers. A failure to receive a heartbeat within a predefined timeout period triggers a failure detection event. Upon detecting a failed node, the system can initiate a recovery procedure. This could involve pausing training temporarily, or, in a more resilient design, continuing training with the remaining healthy nodes while marking the failed node's tasks (e.g., data batches or model shards) as lost. When the failed node comes back online, it can request new tasks from the manager.

Furthermore, the fault tolerance model should be integrated with the parallelism strategy. In a data parallel setup, the loss of a worker is relatively benign; it simply means that one of the data replicas was lost. The training can continue with the remaining replicas, and the failed worker can later be brought back up to rejoin the training cohort. In a model-sharded setup, the consequences of a node failure are more severe, as it can break the computational pipeline. Here, a more sophisticated recovery mechanism is needed. This might involve recomputing the lost activations on the surviving nodes, a

technique known as recomputation-based fault tolerance. While computationally expensive, it avoids the need for storing intermediate states. Lightweight diagnostic approaches based on spiking neural networks have been proposed for edge-side fault diagnosis, which could be adapted to predict or identify impending hardware failures before they cause a crash [57](#) . Other research focuses on adaptive fault tolerance mechanisms that use digital twins to simulate and prepare for potential failures, though this adds significant complexity [61](#) .

The table below outlines key fault tolerance strategies and their relevance to the project's context.

Fault Tolerance Strategy	Description	Suitability for Project
Frequent Global Checkpointing	All nodes save their state to a shared location synchronously at regular intervals.	Unsuitable. Creates I/O bottlenecks and requires shared storage, which contradicts the decentralized nature. 65 81
Asynchronous Per-Node Checkpointing	Each worker saves its own state to local storage periodically.	Highly Suitable. Reduces I/O pressure and enables independent recovery. Aligns with lightweight design principles. 64 81
Recomputation on Failure	Lost intermediate states (e.g., activations) are recomputed by survivors instead of being stored.	Suitable for Model Sharding. Avoids storage overhead but increases computation time. 64
Heartbeat-Based Failure Detection	Nodes periodically send signals to detect unresponsive peers.	Highly Suitable. Simple, lightweight, and effective for detecting node churn in a P2P network. 16 18
Decentralized Aggregation	Model updates are aggregated without a central server, enhancing resilience.	Highly Suitable. Eliminates the single point of failure inherent in a strict master-worker model. 60 62

Ultimately, the goal is to build a system that is "forgiving" of imperfections. By combining asynchronous checkpointing, lightweight failure detection, and a decentralized control model, the fork of nanochat can achieve a level of resilience that makes long-running training jobs feasible on unreliable, consumer-grade hardware.

Managing Hardware Heterogeneity and Resource Optimization

The vision of training LLMs on "slow computers" inherently implies operating in an environment of hardware heterogeneity. Machines connected via a LAN or WAN will almost certainly have varying CPU architectures, clock speeds, amounts of RAM, and storage I/O performance [26](#) [84](#) . Attempting to run a monolithic, synchronous training job

on such a cluster is inefficient and often impossible, as the slowest node (the "straggler") will dictate the pace of the entire process, leading to underutilization of faster machines [71](#). Therefore, the fork must incorporate mechanisms to recognize and adapt to this heterogeneity to maximize overall throughput and ensure fair resource utilization.

The first step in managing heterogeneity is to gather information about the capabilities of each participating node. The system should include a lightweight profiling phase that runs when a node joins the cluster. This profiler can measure key performance indicators such as single-threaded floating-point performance, memory bandwidth, and network round-trip time. This data can then be reported to a central manager or shared among peers in a decentralized manner. With this information, the system can make more informed scheduling decisions. For example, faster nodes can be assigned more computationally intensive tasks, or a larger share of the data batches in a data parallel setup. Conversely, slower nodes can be given lighter tasks or fewer batches. Research in heterogeneous distributed training has explored parameter synchronization models specifically designed for such environments, demonstrating the feasibility of this approach [85](#). More advanced systems even employ schedulers based on asynchronous graph reinforcement learning to dynamically allocate resources, though a simpler heuristic-based approach would be a practical starting point [96](#).

In the context of model sharding, heterogeneity presents another set of challenges. A static partitioning of the model might place computationally heavy layers on slow nodes, creating a new bottleneck. A dynamic sharding strategy could be developed where the model is initially partitioned based on estimated computational load, and then rebalanced over time based on observed performance. This is a complex area of research, but even a basic form of awareness, such as trying to keep computationally similar layers on machines with similar performance characteristics, can yield significant benefits. Some frameworks group resource-limited participants together to jointly instantiate a replica of the model using techniques like pipeline parallelism, which can be an effective way to balance the load [27](#).

Beyond managing heterogeneity, the entire system must be optimized for the constraints of low-resource CPUs. This begins with the choice of the base model architecture. While nanocat's architecture is already simplified, the fork could explore even smaller variants or different architectural choices known to be efficient on CPUs [44](#). The training process itself should rely heavily on highly optimized numerical computing libraries. NumPy is a cornerstone of the scientific Python ecosystem and provides fast, vectorized operations that are essential for efficient matrix computations on CPUs [40](#). All inter-node communication, whether for gradient synchronization or task assignment, must use a

compact serialization format like Protocol Buffers (Protobuf) to minimize CPU cycles spent on encoding and decoding data [94](#). Using binary formats instead of text-based ones like JSON is a critical optimization for reducing communication overhead.

Furthermore, the training loop itself should be optimized for CPU efficiency. This includes using efficient data loading pipelines to feed data to the training workers without becoming a bottleneck. Techniques like prefetching and batching data in advance can help keep the CPU cores busy. The use of mixed-precision training is not applicable here since the focus is on pure CPU training, but other optimizations related to memory access patterns and cache efficiency are still relevant. The system should also be mindful of memory usage. Large models and datasets can easily exhaust the RAM of a single machine. The implementation must carefully manage memory, potentially offloading parts of the model or data to disk if necessary, although this will come at a significant performance penalty. Lightweight IPC mechanisms and novel resource amplification methods, as seen in projects like Pocket, could inspire ways to more effectively utilize the limited resources available on edge devices [8](#).

Dashboard Implementation and Real-Time Monitoring Strategies

The integration of an HTML-based dashboard is a key requirement for providing visibility into the distributed training process. This dashboard serves as the primary interface for users to monitor progress, diagnose issues, and manage the training job running across multiple machines. To align with the project's goal of being accessible on low-resource computers, the implementation of the dashboard must prioritize efficiency and simplicity. This involves making thoughtful choices regarding the backend web framework and the real-time communication protocol used to push updates from the training nodes to the user's browser.

For the backend, a heavyweight, full-stack web framework like Django is unnecessary and would consume excessive resources [31](#). Instead, a micro-framework is the ideal choice. Flask is a popular option known for its minimalist design, which makes it easy to integrate into existing applications and perfect for building small APIs and web services [31](#) [32](#). FastAPI is another compelling alternative, offering modern features like automatic interactive API documentation and high performance through its asynchronous, non-blocking architecture based on Python's `asyncio` [78](#). Its ability to handle many

concurrent I/O operations efficiently makes it well-suited for a dashboard that needs to serve multiple clients and frequently poll for updated metrics. Bottle is another single-file micro-framework noted for its extremely low memory footprint, making it a strong contender for the most resource-constrained environments [48](#). Given that nanochat is written in Python, integrating one of these frameworks would be seamless and require minimal additional dependencies.

The most critical aspect of the dashboard's implementation is the real-time communication mechanism. The dashboard needs to display live-updating metrics such as training loss, validation accuracy, throughput (tokens/second), and the status of individual worker nodes. Several technologies exist for pushing data from the server to the client without requiring the client to constantly refresh the page.

Communication Technology	Description	Pros	Cons	Best Use Case
HTTP Polling / Long-Polling	Client repeatedly requests data from the server. Long-polling keeps the connection open until data is ready.	Simple to implement. Widely supported.	High latency and overhead. Generates significant network traffic, especially with frequent polling. Not efficient. 3 4	Low-frequency updates where overhead is acceptable. Generally discouraged for real-time apps. 47
Server-Sent Events (SSE)	A server pushes data to the client over a standard HTTP connection. Data flow is unidirectional (server-to-client).	Efficient bandwidth usage. Simpler than WebSockets. Standard HTTP, so less likely to be blocked by firewalls.	Unidirectional only. Less feature-rich than WebSockets.	Ideal for dashboards, logs, and other monitoring interfaces where data flows primarily from server to client. 3 6
WebSockets	Establishes a persistent, full-duplex (bidirectional) communication channel between the client and server.	Very low latency. Supports bidirectional communication.	Higher overhead than SSE. Connection can be complex to manage. May be blocked by restrictive firewalls. 30 46	Applications requiring frequent, bidirectional interaction (e.g., chat, gaming). Overkill for a simple monitor.

Based on this comparison, Server-Sent Events (SSE) emerge as the most suitable technology for this project. The primary use case—a monitoring dashboard—is a classic example of a server-to-client data streaming scenario. SSE is designed for exactly this purpose and is more bandwidth-efficient than traditional HTTP polling, as the connection remains open and data is transmitted only when an event occurs [3](#) [6](#). It leverages standard HTTP, making it more likely to traverse firewalls and proxies compared to WebSockets [47](#). While WebSockets offer bidirectional communication, the need for the dashboard to send control commands back to the training nodes is not specified and can be handled via separate, standard HTTP POST requests if necessary. Therefore, SSE provides the best combination of performance, efficiency, and simplicity for the stated goal.

The implementation would involve creating a backend endpoint (e.g., `/metrics`) that the training nodes or the manager process can write to. This endpoint would be an SSE stream. When a metric like the current loss value changes, the process would simply write a line in the format `data: {"loss": 2.13, "timestamp": ...}` to the response stream for all connected clients. The frontend JavaScript would establish an `EventSource` connection to this endpoint and listen for messages, updating the corresponding chart or display element in real-time. This approach is lightweight, robust, and perfectly aligned with the project's emphasis on minimalism and efficiency. Plotly, a library mentioned in the context of dashboards, can be used to create interactive charts that can be easily populated with data pushed via SSE [39](#).

Reference

1. Distributed training of multiple models on multiple nodes (CPU only) <https://discuss.pytorch.org/t/distributed-training-of-multiple-models-on-multiple-nodes-cpu-only/33930>
2. Run distributed inference with llama.cpp on Arm-based AWS ... <https://learn.arm.com/learning-paths/servers-and-cloud-computing/distributed-inference-with-llama-cpp/>
3. HTTP Polling VS SSE - PieHost <https://piehost.com/websocket/http-polling-vs-sse>
4. What are Long-Polling, Websockets, Server-Sent Events (SSE) and ... <https://stackoverflow.com/questions/11077857/what-are-long-polling-websockets-server-sent-events-sse-and-comet>
5. Low-latency Video Streaming - IEEE Xplore <https://ieeexplore.ieee.org/iel8/9739/5451756/11501921.pdf>
6. Server-Sent Events Beat WebSockets for 95% of Real-Time Apps ... <https://dev.to/polliog/server-sent-events-beat-websockets-for-95-of-real-time-apps-heres-why-a4l>
7. Lightweight multi-hop routing protocol for resource optimisation in ... <https://www.sciencedirect.com/science/article/pii/S2542660523000811>
8. Pocket: ML Serving from the Edge - ACM Digital Library <https://dl.acm.org/doi/10.1145/3552326.3587459>
9. [PDF] Lightweight Latency Prediction Scheme for Edge Applications - arXiv <https://arxiv.org/pdf/2511.02501>

10. Lightweight Signal Processing and Edge AI for Real-Time Anomaly ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC12610206/>
11. Efficient Training of Large Language Models on Distributed ... - arXiv <https://arxiv.org/html/2407.20018v1>
12. Distributed training of large language models: A survey <https://www.sciencedirect.com/science/article/pii/S2949719125000500>
13. [PDF] Beyond A Single AI Cluster: A Survey of Decentralized LLM Training <https://aclanthology.org/2025.emnlp-main.1486.pdf>
14. (PDF) Efficient Training of Large Language Models on Distributed ... https://www.researchgate.net/publication/382654730_Efficient_Training_of_Large_Language_Models_on_Distributed_Infrastructures_A_Survey
15. How to Set Up Cloud-Based Distributed Training to Fine-Tune an LLM <https://www.intel.com/content/www/us/en/developer/articles/technical/set-up-cloud-based-distributed-training.html>
16. Decentralized formation control with connectivity maintenance and ... https://www.researchgate.net/publication/257247227_Decentralized_formation_control_with_connectivity_maintenance_and_collision_avoidance_under_limited_and_intermittent_sensing
17. Agentic Patterns for Decentralized Network Protocol Configuration <https://www.mdpi.com/2079-9292/15/11/2270>
18. Decentralized Multi-Robot Line-of-Sight Connectivity Maintenance ... <https://arxiv.org/abs/2406.12802>
19. Adaptive WAN Link Anomaly Detection Using Lightweight Packet ... <https://ijsrmt.com/index.php/ijsrmt/article/view/1348>
20. Scaling LLM fine-tuning with sharding - IBM Developer <https://developer.ibm.com/articles/llms-sharding/>
21. Learning to Shard: RL for Co-optimizing the Parallelism Degrees ... <https://arxiv.org/html/2509.00217v1>
22. Lightweight and Fast Error Recovery for LLM Training in a Just-In ... <https://dl.acm.org/doi/10.1145/3735358.3735372>
23. Light-Weight Fault Tolerant Attention for Large Language Model ... <https://arxiv.org/html/2410.11720v1>
24. TRANSOM: An Efficient Fault-Tolerant System for Training LLMs <https://www.semanticscholar.org/paper/TRANSOM%3A-An-Efficient-Fault-Tolerant-System-for-Wu-Xia/ffa343b5dce9e82f00fbf380f8b343963109f8ec>
25. [PDF] A Lightweight Tool to Automate Distributed Training of LLMs on Any ... <https://aclanthology.org/2024.naacl-demo.14.pdf>

26. Training Large Models on Heterogeneous and Geo-Distributed ... <http://ieeexplore.ieee.org/iel8/8254253/11002434/11002440.pdf>
27. Heterogeneous Low-Bandwidth Pre-Training of LLMs - OpenReview <https://openreview.net/forum?id=OVhp2mHAPW>
28. Decoding the Magic Behind Andrej Karpathy's NanoChat - LinkedIn <https://www.linkedin.com/pulse/decoding-magic-behind-andrej-karpathys-nanochat-prashant-lakhera-ypbsc>
29. My homelab needed a lightweight dashboard, so I had Claude and ... <https://www.xda-developers.com/my-homelab-needed-a-lightweight-dashboard-so-i-had-claude-and-a-local-llm-race-to-build-it/>
30. WebSockets vs Server-Sent Events: Key differences and which to use <https://ably.com/blog/websockets-vs-sse>
31. Which Is the Best Python Web Framework: Django, Flask, or FastAPI? <https://blog.jetbrains.com/pycharm/2025/02/django-flask-fastapi/>
32. Alternatives, Inspiration and Comparisons - FastAPI <https://fastapi.tiangolo.com/alternatives/>
33. Efficient training of large language models on distributed ... <https://link.springer.com/article/10.1007/s44336-026-00038-z>
34. Network Edge Inference for Large Language Models: Principles ... <https://dl.acm.org/doi/10.1145/3809166>
35. AI/ML Model Cards in Edge AI Cyberinfrastructure: towards Agentic AI <https://arxiv.org/html/2511.21661v1>
36. Combining Machine Learning and Edge Computing - MDPI <https://www.mdpi.com/2079-9292/13/3/640>
37. Machine Learning on Low-Cost Edge Devices for Real-Time Water ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC12526668/>
38. Are there any lightweight analogues to CORBA/RPC for embedded ... <https://stackoverflow.com/questions/2656487/are-there-any-lightweight-analogues-to-corba-rpc-for-embedded-programs>
39. How to export a plotly dashboard app into a html standalone file to ... <https://stackoverflow.com/questions/60097577/how-to-export-a-plotly-dashboard-app-into-a-html-standalone-file-to-share-with-t>
40. scikit-learn: machine learning in Python — scikit-learn 1.9.0 ... <http://scikit-learn.org/>
41. Get Exclusive ChatGPT in 4 Hours: Karpathy's Old Tricks, 8,000 ... <https://eu.36kr.com/en/p/3508936344689539>
42. Simulating LLM training workloads for heterogeneous compute and ... <https://arxiv.org/html/2508.05370v1>

43. Efficiently Scale LLM Training Across a Large GPU Cluster with Alpa ... <https://developer.nvidia.com/blog/efficiently-scale-llm-training-across-a-large-gpu-cluster-with-alpa-and-ray/>
44. Efficient LLMs for CPU-Only Usage? - Kaggle <https://www.kaggle.com/questions-and-answers/549854>
45. (PDF) Deploying LLMs on CPU-only Environments with llama.cpp ... https://www.researchgate.net/publication/400783182_Deploying_LLMs_on_CPU-only_Environments_with_llamacpp_Library_Set_MedLocalGPT_Project_Case
46. At what point are WebSockets less efficient than Polling? <https://stackoverflow.com/questions/44731313/at-what-point-are-websockets-less-efficient-than-polling>
47. WebSocket vs Polling vs SSE: An Architect's Perspective - LinkedIn <https://www.linkedin.com/pulse/websocket-vs-polling-sse-architects-perspective-kishor-naik-2denc>
48. Is there a python webframework which is able to run decently on ... <https://stackoverflow.com/questions/11812690/is-there-a-python-webframework-which-is-able-to-run-decently-on-128mb-of-ram>
49. karpathy/autoresearch: AI agents running research on single-GPU ... https://www.linkedin.com/posts/niallm_github-karpathyautoresearch-ai-agents-activity-7437230871096573952-GnH9
50. 大神卡帕西最疯狂开源项目！nanochat最快4小时100美元克隆ChatGPT <https://zhuanlan.zhihu.com/p/1961816125022122395>
51. 8k lines of code, nanochat: > Trains a... | Zain Hasan - LinkedIn https://www.linkedin.com/posts/zainhas_new-nanochat-repo-from-karpathy-its-a-minimal-activity-7383527694958964736-SfbM
52. Best Python supported server/client protocol? - Stack Overflow <https://stackoverflow.com/questions/64426/best-python-supported-server-client-protocol>
53. Privacy-Preserving and Lightweight Selective Aggregation with Fault ... <https://www.mdpi.com/1424-8220/21/16/5369>
54. Privacy-Preserving and Lightweight Selective Aggregation with Fault ... <https://pubmed.ncbi.nlm.nih.gov/34450808/>
55. [PDF] DRAGON: Decentralized Fault Tolerance in Edge Federations - arXiv <https://arxiv.org/pdf/2208.07658>
56. Edge Computing-based Fault-Tolerant Framework: A Case Study on ... https://www.researchgate.net/publication/343249974_Edge_Computing-based_Fault-Tolerant_Framework_A_Case_Study_on_Vehicular_Networks
57. The Lightweight Edge-Side Fault Diagnosis Approach Based on ... <https://www.sciopen.com/article/10.32604/cmc.2024.051860>

58. (PDF) Heterogeneous Low-Bandwidth Pre-Training of LLMs https://www.researchgate.net/publication/399477355_Heterogeneous_Low-Bandwidth_Pre-Training_of_LLMs
59. Simulating LLM training workloads for heterogeneous compute and ... <https://dl.acm.org/doi/pdf/10.1145/3748273.3749212>
60. Enabling efficient and low-effort decentralized federated learning ... <https://www.sciencedirect.com/science/article/pii/S0950584924002052>
61. Adaptive fault tolerance mechanisms for ensuring high availability of ... <https://www.nature.com/articles/s41598-025-25590-4>
62. Fault-Tolerant Decentralized Distributed Asynchronous Federated ... https://www.researchgate.net/publication/395213061_Fault-Tolerant_Decentralized_Distributed_Asynchronous_Federated_Learning_with_Adaptive_Termination_Detection
63. Fault-Tolerance in the Scope of Cloud Computing - IEEE Xplore <https://ieeexplore.ieee.org/iel7/6287639/9668973/09793702.pdf>
64. All is Not Lost: LLM Recovery without Checkpoints - arXiv <https://arxiv.org/html/2506.15461v2>
65. All is Not Lost: LLM Recovery without Checkpoints - arXiv <https://arxiv.org/html/2506.15461v1>
66. [PDF] Boosting Distributed Machine Learning Training Through Loss ... <https://arxiv.org/pdf/2305.04279>
67. Beyond 5G: PHWAN – A secure, low-latency, and cost-effective ... <https://www.sciencedirect.com/science/article/pii/S111086652500252X>
68. Field Trial of Multi-Datacenter Distributed Training for LLM Based on ... https://www.researchgate.net/publication/392394314_Field_Trial_of_Multi-Datacenter_Distributed_Training_for_LLM_Based_on_Bandwidth_Convergence_and_Two_Parallel_Strategies_over_120km_High-reliability_800Gbits_CL_OTN
69. 百元级「最好的ChatGPT」，Karpathy 的极简LLM 训练套件- 掘金 <https://juejin.cn/post/7606229757463085107>
70. 成本压到50 美元! nanochat 重新定义「轻量级大模型训练」 <https://www.woshipm.com/ai/6383108.html>
71. Chuan Wu: Publications <https://i.cs.hku.hk/~cwu/publications.html>
72. Unpacking distributed machine learning: A unified taxonomy, formal ... <https://www.sciencedirect.com/science/article/pii/S0925231226002250>
73. Training an Open-Source LLM on Windows (CPU-Only) - LinkedIn <https://www.linkedin.com/pulse/training-opensource-llm-windows-cpuonly-manish-ojha-fdvgc>

74. Distributed Computing with Language Models at the Edge <https://dl.acm.org/doi/10.1145/3770501.3770511>
75. [PDF] A CPU/GPU Heterogeneous Speculative Decoding for LLM inference <https://aclanthology.org/2025.emnlp-main.879.pdf>
76. (PDF) P2PLLMEdge: Peer-to-Peer Framework for Localized Large ... https://www.researchgate.net/publication/393627446_P2PLLMEdge_Peer-to-Peer_Framework_for_Localized_Large_Language_Models_using_CPU_only_Resource-Constrained_Edge
77. Server-Sent Events (SSE) vs WebSockets vs Long Polling <https://dev.to/haraf/server-sent-events-sse-vs-websockets-vs-long-polling-whats-best-in-2025-5ep8>
78. 2026 Python Web 框架终极对比：一篇看懂Django/Flask/FastAPI ... https://blog.csdn.net/qq_28372005/article/details/160161457
79. SHIFT: An RDMA Failure-Resilient Layer for Distributed Training <https://arxiv.org/html/2512.11094v1>
80. Enabling Fast Inference and Resilient Training with NCCL 2.27 <https://developer.nvidia.com/blog/enabling-fast-inference-and-resilient-training-with-nccl-2-27/>
81. (PDF) DataStates-LLM: Lazy Asynchronous Checkpointing for Large ... https://www.researchgate.net/publication/381485419_DataStates-LLM_Lazy_Asynchronous_Checkpointing_for_Large_Language_Models
82. A lightweight fault tolerance framework for Web services https://www.researchgate.net/publication/220298486_A_lightweight_fault_tolerance_framework_for_Web_services
83. Micro-FL: A Fault-Tolerant Scalable Microservice-Based Platform for ... <https://www.mdpi.com/1999-5903/16/3/70>
84. Rail-only: A Low-Cost High-Performance Network for Training LLMs ... https://www.researchgate.net/publication/383936016_Rail-only_A_Low-Cost_High-Performance_Network_for_Training_LLMs_with_Trillion_Parameters
85. Distributed Machine Learning through Heterogeneous Edge Systems <https://ojs.aaai.org/index.php/AAAI/article/view/6207>
86. Paper Sessions https://ccf.org.cn/ICSS2026/general_3052
87. Journal of Computer Research and Development 2026 Vol. 63 No. 6 <https://crad.ict.ac.cn/en/article/current>
88. Distributed RPC Framework — PyTorch 2.12 documentation <https://docs.pytorch.org/docs/stable/rpc.html>
89. Leveraging decentralized communication for privacy-preserving ... <https://www.sciencedirect.com/science/article/pii/S0140366425000295>

90. Python Web 开源框架排行榜- klchang - 博客园 <https://www.cnblogs.com/klchang/p/18026572>
91. Performance difference between websocket and server sent events ... <https://stackoverflow.com/questions/63583989/performance-difference-between-websocket-and-server-sent-events-sse-for-chat-r>
92. HetCCL: Accelerating LLM Training with Heterogeneous GPUs - arXiv <https://arxiv.org/html/2601.22585v1>
93. tinyrpc: A modular RPC library — tinyrpc 1.1.7 documentation <https://tinyrpc.readthedocs.io/>
94. connect-python - PyPI <https://pypi.org/project/connect-python/>
95. Is python grpc client support retry? - Stack Overflow <https://stackoverflow.com/questions/65771439/is-python-grpc-client-support-retry>
96. Fault-Tolerant Scheduling Mechanism for Dynamic Edge Computing ... <https://www.mdpi.com/1424-8220/24/21/6984>
97. Long Polling vs WebSockets: What's best for realtime at scale? <https://ably.com/blog/websockets-vs-long-polling>
98. Nanochat Training | DGX Station <https://build.nvidia.com/station/nanochat/instructions>
99. Heterogeneous Confidential Computing System for Large Language ... <https://dl.acm.org/doi/10.1145/3779307>
100. Chuan Wu: Research <https://i.cs.hku.hk/~cwu/research.html>